Proyecto Base de Datos: Gestión de Recolección de Aceitunas

Índice

1. Descripción de la temática del proyecto **3**
2. Resolución del diagrama MER extendido **3**
2.1. Entidades *4*
2.2. Relaciones *5*
3. La transformación a Modelo Relacional 5
4. Carga Masiva de Datos **6**
5. Consultas Multitabla SQL **13**
5.1 Consulta 1 – (4 tablas) *13*
5.2 Consulta 2 – GROUP BY *13*
5.3 Consulta 3 – Subconsulta *13*
5.4 Consulta 4 – Funciones de fecha *13*
5.5 Consulta 5 – Funciones de texto *14*
5.6 Consulta 6 – ISNULL / ISNULL *14*
5.7 Consulta 7 – Consulta compleja *14*
6. Vistas **15**
6.1 Vista 1 ..... *15*
6.2 Vista 2 ..... *15*
7. Scripts SQL del Proyecto **16**
8. Conclusión y valoración personal Parte 1 ..... **17**

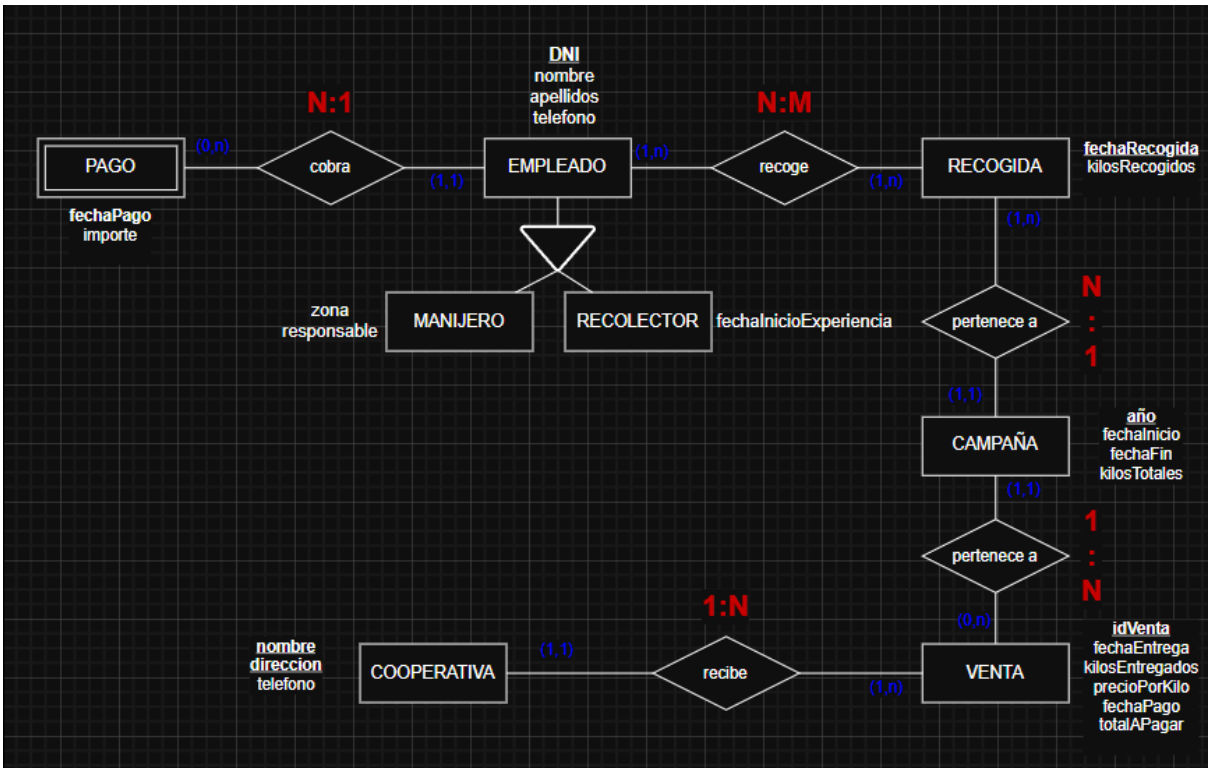
1. Descripción de la temática del proyecto.

He hecho este proyecto porque tengo una finca con olivos y quería crear una base de datos que sirviera para organizar la recogida de aceitunas.

El sistema permite registrar los empleados, las campañas, los pagos, las cooperativas y las ventas, para llevar un control más claro y automatizado.

Con este diseño se puede saber quién trabaja, cuánto se recoge y cuánto se paga o se vende, ayudando a gestionar mejor cada campaña.

2. Resolución del diagrama MER extendido



2.1. Entidades

Entidad EMPLEADO (Padre de la Herencia)

Guarda los datos personales del trabajador (**DNI, nombre, apellidos y teléfono**).

Un empleado puede ser de dos tipos y puede ser los 2 a la vez:

- **MANIJERO (Herencia)**, que tiene una zona a su cargo (Tiene que controlar a los recolectores y decir hoy se coje en tal zona y tales olivos y se para a tal hora).
- **RECOLECTOR (Herencia)**, que tiene un año para saber en qué año empezó a recoger aceitunas.

Entidad PAGO (Débil)
Registra los pagos realizados a los empleados, con su **fecha** e **importe**.

Entidad RECOGIDA
Guarda los datos de cada recogida (**fecha y kilos recogidos**).

Entidad CAMPAÑA
Representa cada temporada de trabajo (**año, fechas de inicio y fin, kilos totales** recogidos de cada campaña).

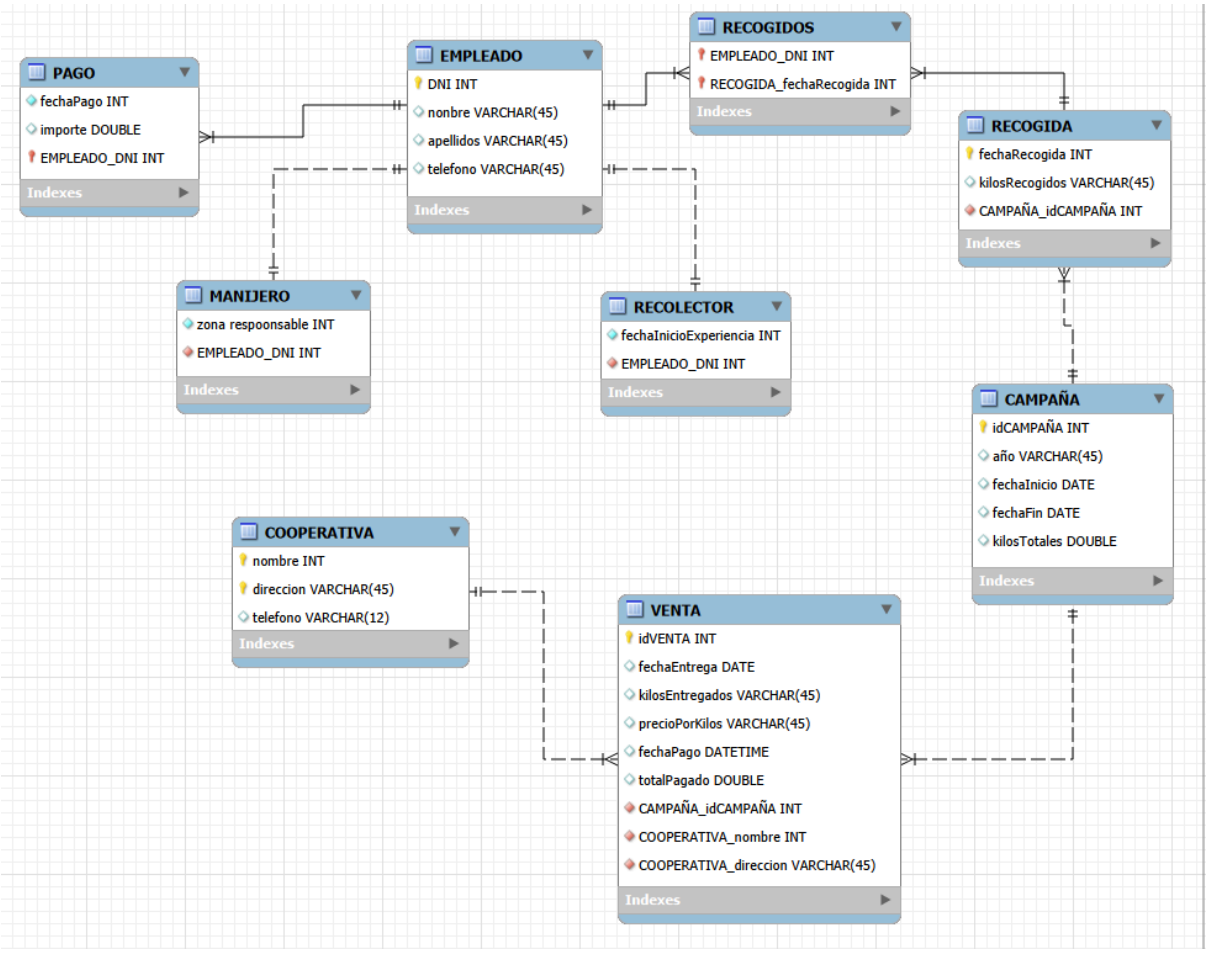
Entidad VENTA
Registra las ventas de aceitunas (**idVenta, fecha, kilos entregados y precio por kilo, fechaPago, totalAPagar**).

Entidad COOPERATIVA
Guarda los datos de las cooperativas a las que se entregan las aceitunas (**nombre, dirección y teléfono**).

2.2. Relaciones

- **PAGO – EMPLEADO (N:1):** Un empleado puede tener muchos pagos, pero un pago pertenece a un solo empleado.
- **EMPLEADO – RECOGIDA (N:M):** Ya que varios empleados pueden participar en distintas recogidas.
- **RECOGIDA – CAMPAÑA (N:1):** Una campaña incluye varias recogidas, y una recogida pertenece a una sola campaña.
- **CAMPAÑA – VENTA (1:N):** Cada campaña puede tener varias ventas, pero una venta pertenece a una sola campaña.
- **VENTA – COOPERATIVA (N:1):** Una cooperativa puede recibir muchas ventas, pero una venta es a solamente a una cooperativa.

3. La transformación a Modelo Relacional

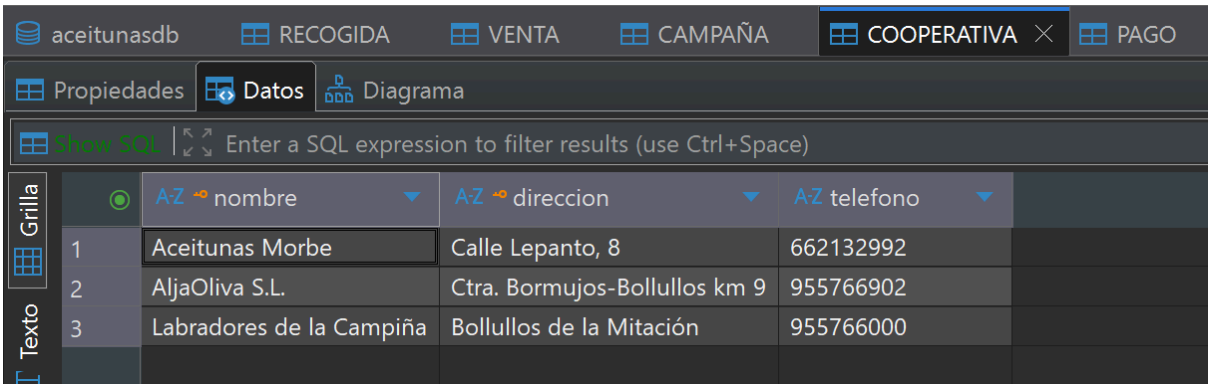


4. Carga Masiva de Datos

Para completar este apartado, he utilizado Gemini AI (no vale para nada e tenido que modificar muchísimo incluso tabla de pagos hacer yo mismo el csv poco a poco con calculadora para calcular el dinero y de todo) como apoyo para generar gran parte de los datos. Al haberle facilitado previamente la estructura de la base de datos, ha sido más sencillo obtener archivos CSV con información coherente, como nombres reales, DNI que encajan entre tablas y fechas lógicas para una campaña agrícola.

En el caso de la tabla COOPERATIVA, he optado por usar SQL por qué son 3 cooperativas, con estas líneas de código:

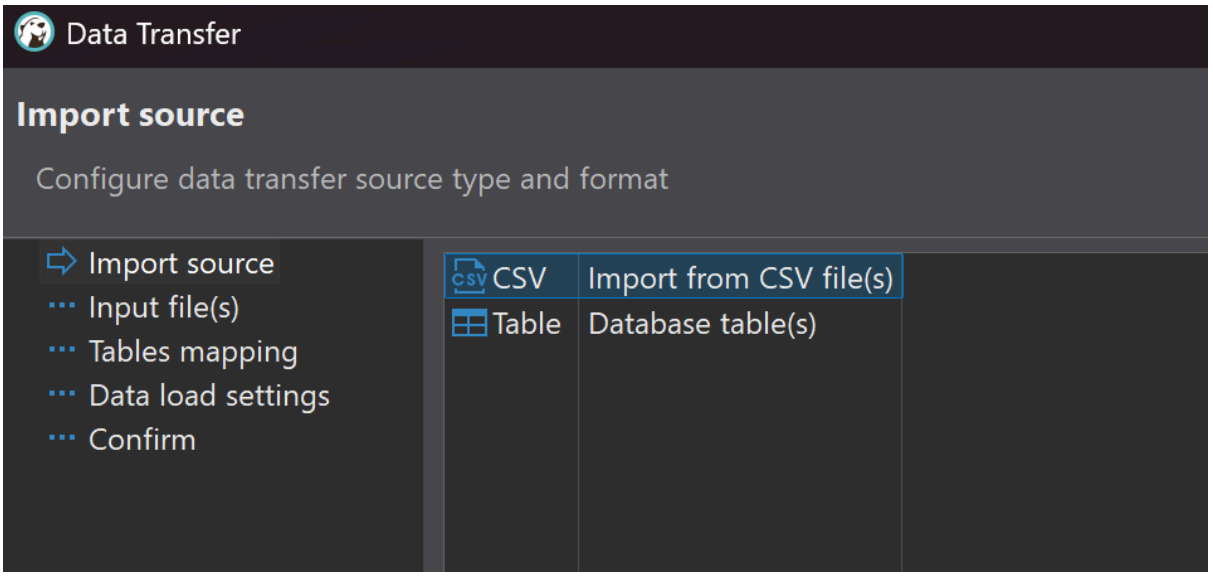
```
INSERT INTO `aceitunasdb`.`COOPERATIVA` (nombre, direccion, telefono) VALUES
('AljaOliva S.L.', 'Ctra. Bormujos-Bollullos km 9', 955766902),
('Aceitunas Morbe', 'Calle Lepanto, 8', 662132992),
('Labradores de la Campiña', 'Bollullos de la Mitación', 955766000);
```



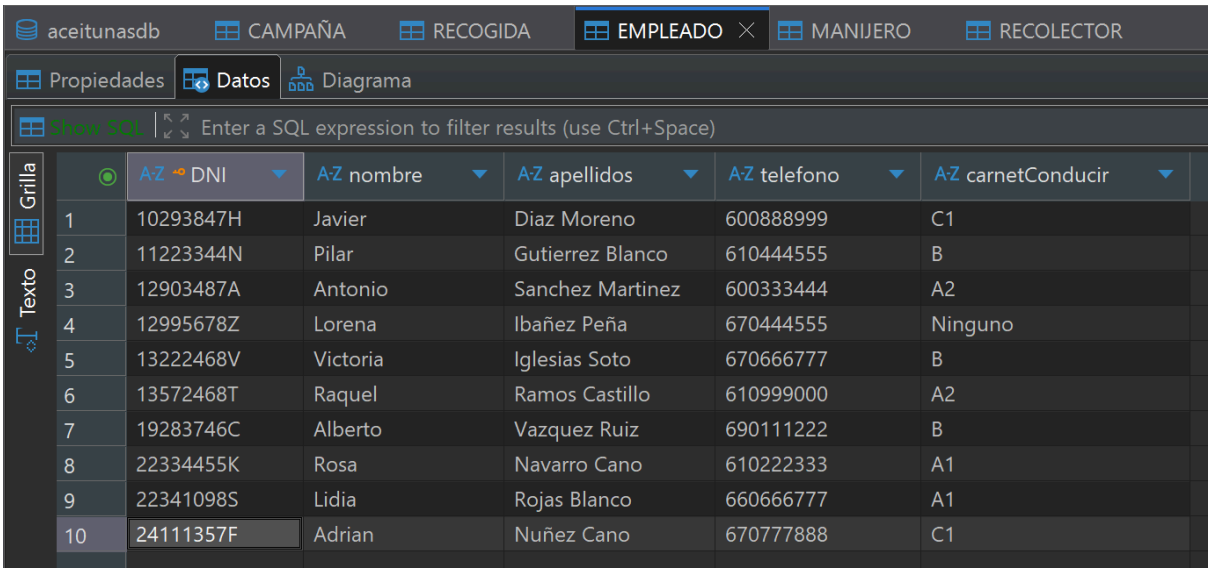
Finalmente, toda la información se ha metido en la base de datos de AWS usando el asistente de importación de DBeaver, revisando que se mantuvieran las claves foráneas.

Hago un paso a paso de como e importado la primera tabla y ya todos así:

1. Nos descargamos los CSV correspondientes.
2. Nos vamos al DBeaver a la tabla que queramos importarles los datos.
3. Click derecho a la tabla > Import Data > CSV:



4. Le damos a siguiente y seleccionamos el archivo desde nuestro explorador de archivos.
5. Le damos a todo “Siguiente” y por último a “Continuar”, y se meterán todos los datos:



	DNI	nombre	apellidos	telefono	carnetConducir
1	10293847H	Javier	Diaz Moreno	600888999	C1
2	11223344N	Pilar	Gutierrez Blanco	610444555	B
3	12903487A	Antonio	Sanchez Martinez	600333444	A2
4	12995678Z	Lorena	Ibañez Peña	670444555	Ninguno
5	13222468V	Victoria	Iglesias Soto	670666777	B
6	13572468T	Raquel	Ramos Castillo	610999000	A2
7	19283746C	Alberto	Vazquez Ruiz	690111222	B
8	22334455K	Rosa	Navarro Cano	610222333	A1
9	22341098S	Lidia	Rojas Blanco	660666777	A1
10	24111357F	Adrian	Nuñez Cano	670777888	C1

Seguidamente hacemos la de Manijeros:

aceitunasdb | CAMPAÑA | RECOGIDA | EMPLEADO | MANIJERO

Propiedades | Datos | Diagrama

Show SQL | Enter a SQL expression to filter results (use Ctrl+Space)

	zonaResponsable	EMPLEADO_DNI
1	Finca El Pintao - Sector Norte	10293847H
2	Finca El Pintao - Sector Sur	11223344N

Y la de recolectores:

aceitunasdb | CAMPAÑA | RECOGIDA | EMPLEADO | MANIJERO | RECOLECTOR

Propiedades | Datos | Diagrama

Show SQL | Enter a SQL expression to filter results (use Ctrl+Space)

	fechaInicioExperiencia	EMPLEADO_DNI
1	1988	12903487A
2	2008	12995678Z
3	2003	13222468V
4	2002	13572468T
5	1998	19283746C
6	2005	22334455K
7	2001	22341098S
8	1995	24111357F

La de campaña:

aceitunasdb | CAMPAÑA | RECOGIDA | EMPLEADO | MANIJERO | RECOLECTOR | RECO

Propiedades | Datos | Diagrama

Show SQL | Enter a SQL expression to filter results (use Ctrl+Space)

	idCampaña	año	fechaInicio	fechaFin	kilosTotales
1	1	2023	2023-07-10	2023-12-31	730,736
2	2	2024	2024-07-01	2024-12-20	1.111.112
3	3	2025	2025-07-01	2025-11-20	750.000

La de recogida que están todos los días 1 a 1 desde fecha inicio a fecha fin que se indica en campaña:

aceitunasdb | CAMPAÑA | RECOGIDA | EMPLEADO | MANIJERO | RECOL

Propiedades | Datos | Diagrama

Show SQL | Enter a SQL expression to filter results (use Ctrl+Space)

Grilla | Texto

	fechaRecogida	kilosRecogidos	CAMPAÑA_idCAMPAÑA
1	2023-07-10	3.970	1
2	2023-07-11	3.975	1
3	2023-07-12	3.965	1
4	2023-07-13	3.980	1
5	2023-07-14	3.972	1
6	2023-07-15	3.973	1
7	2023-07-16	3.980	1
8	2023-07-17	3.967	1
9	2023-07-18	3.975	1
10	2023-07-19	3.970	1
11	2023-07-20	3.975	1
12	2023-07-21	3.965	1
13	2023-07-22	3.980	1
14	2023-07-23	3.972	1
15	2023-07-24	3.973	1
16	2023-07-25	3.980	1
17	2023-07-26	3.967	1
18	2023-07-27	3.975	1
19	2023-07-28	3.970	1
20	2023-07-29	3.975	1
21	2023-07-30	3.965	1
22	2023-07-31	3.980	1
23	2023-08-01	3.972	1
24	2023-08-02	3.973	1
25	2023-08-03	3.980	1

(491 Filas)

Ahora con recogidos:

aceitunasdb | RECOGIDOS

Propiedades | Datos | Diagrama

Show SQL | Enter a SQL expression to filter results (use Ctrl+Space)

Grilla | Texto

	EMPLEADO_DNI	RECOGIDA_fechaRecogida
1	10293847H	2023-07-10
2	11223344N	2023-07-10
3	12903487A	2023-07-10
4	12995678Z	2023-07-10
5	13222468V	2023-07-10
6	13572468T	2023-07-10
7	19283746C	2023-07-10
8	22334455K	2023-07-10
9	22341098S	2023-07-10
10	24111357F	2023-07-10
11	10293847H	2023-07-11
12	11223344N	2023-07-11
13	12903487A	2023-07-11
14	12995678Z	2023-07-11
15	13222468V	2023-07-11
16	13572468T	2023-07-11
17	19283746C	2023-07-11
18	22334455K	2023-07-11
19	22341098S	2023-07-11
20	24111357F	2023-07-11
21	10293847H	2023-07-12
22	11223344N	2023-07-12
23	12903487A	2023-07-12
24	12995678Z	2023-07-12
25	13222468V	2023-07-12
26	13572468T	2023-07-12
27	19283746C	2023-07-12
28	22334455K	2023-07-12
29	22341098S	2023-07-12
30	24111357F	2023-07-12
31	10293847H	2023-07-13
32	11223344N	2023-07-13

(3490 Filas)

Ahora la tabla pagos, he pagado 50€ al día y voy a pagar por cada mes, y como todos los trabajadores han ido todos los días pues es más fácil de calcular.

aceitunasdb | RECOGIDA | VENTA | CAMPAÑA | COOPERATIVA | PAGO

Propiedades | Datos | Diagrama

Show SQL | Enter a SQL expression to filter results (use Ctrl+Space)

Grilla | Texto

	EMPLEADO_DNI	fechaPago	importe
1	10293847H	2023-07-31	1.320
2	10293847H	2023-08-31	1.860
3	10293847H	2023-09-30	1.800
4	10293847H	2023-10-31	1.860
5	10293847H	2023-11-30	1.800
6	10293847H	2023-12-31	1.860
7	10293847H	2024-07-31	1.860
8	10293847H	2024-08-31	1.860
9	10293847H	2024-09-30	1.800
10	10293847H	2024-10-31	1.860
11	10293847H	2024-11-30	1.800
12	10293847H	2024-12-20	1.200
13	10293847H	2025-07-31	1.860
14	10293847H	2025-08-31	1.860
15	10293847H	2025-09-30	1.800
16	10293847H	2025-10-31	1.860
17	10293847H	2025-11-20	1.200
18	11223344N	2023-07-31	1.320

(170 Filas)

Y por último la tabla de ventas:
Me daba error al meter el .csv, todavía no entiendo el porque, la IA (Chat GPT), cogió mi .csv y lo convirtió en un SCRIPT SQL sin modificar nada y así me fue:

aceitunasdb | RECOGIDA | CAMPAÑA | COOPERATIVA | PAGO | *<AWS-AceitunasDB> Script-11 | VENTA

Propiedades | Datos | Diagrama

Enter a SQL expression to filter results (use Ctrl+Space)

	idVENTA	fechaEntrega	kilosEntregados	totalPagado	CAMPAÑA_idCAMPAÑA	COOPERATIVA_nombre	COOPERATIVA_direccion
1	1	2023-07-31	87.404	786.636	1	Aceitunas Morbe	Calle Lepanto, 8
2	2	2023-08-31	111.326	1.001.934	1	Aceitunas Morbe	Calle Lepanto, 8
3	3	2023-09-30	1.184.340	1.065.906	1	Aceitunas Morbe	Calle Lepanto, 8
4	4	2023-10-31	123.038	1.107.342	1	AljaOliva S.L.	Ctra. Bormujos-Bollullos km 9
5	5	2023-11-30	118.501	106.650,9	1	AljaOliva S.L.	Ctra. Bormujos-Bollullos km 9
6	6	2023-12-31	118.476	1.066.284	1	AljaOliva S.L.	Ctra. Bormujos-Bollullos km 9
7	7	2024-07-31	201.400	181.260	2	Labradores de la Campiña	Bollullos de la Mitación
8	8	2024-08-31	201.394	1.812.546	2	Labradores de la Campiña	Bollullos de la Mitación
9	9	2024-09-30	192.634	173.370,6	2	Labradores de la Campiña	Bollullos de la Mitación
10	10	2024-10-31	201.014	180.912,6	2	Aceitunas Morbe	Calle Lepanto, 8
11	11	2024-11-30	201.019	180.917,1	2	Aceitunas Morbe	Calle Lepanto, 8
12	12	2024-12-20	128.637	115.773,3	2	Aceitunas Morbe	Calle Lepanto, 8
13	13	2025-07-31	133.020	119.718	3	AljaOliva S.L.	Ctra. Bormujos-Bollullos km 9
14	14	2025-08-31	133.025	119.722,5	3	AljaOliva S.L.	Ctra. Bormujos-Bollullos km 9
15	15	2025-09-30	129.010	116.109	3	AljaOliva S.L.	Ctra. Bormujos-Bollullos km 9
16	16	2025-10-30	138.455	124.609,5	3	Labradores de la Campiña	Bollullos de la Mitación
17	17	2025-11-20	86.270	77.643	3	Labradores de la Campiña	Bollullos de la Mitación

Este es el SCRIPT:

```
INSERT INTO
VENTA(idVENTA,fechaEntrega,kilosEntregados,totalPagado,CAMPAÑA_idCAMPAÑA,COOPERATIVA_nombre,COOPERATIVA_direccion) VALUES
(1,'2023-07-31',87404.0,786636.0,1,'Aceitunas Morbe','Calle Lepanto, 8'),
(2,'2023-08-31',111326.0,1001934.0,1,'Aceitunas Morbe','Calle Lepanto, 8'),
(3,'2023-09-30',1184340.0,1065906.0,1,'Aceitunas Morbe','Calle Lepanto, 8'),
(4,'2023-10-31',123038.0,1107342.0,1,'AljaOliva S.L.','Ctra. Bormujos-Bollullos km 9'),
(5,'2023-11-30',118501.0,106650.9,1,'AljaOliva S.L.','Ctra. Bormujos-Bollullos km 9'),
(6,'2023-12-31',118476.0,1066284.0,1,'AljaOliva S.L.','Ctra. Bormujos-Bollullos km 9'),
(7,'2024-07-31',201400.0,181260.0,2,'Labradores de la Campiña','Bollullos de la Mitación'),
(8,'2024-08-31',201394.0,1812546.0,2,'Labradores de la Campiña','Bollullos de la Mitación'),
(9,'2024-09-30',192634.0,173370.6,2,'Labradores de la Campiña','Bollullos de la Mitación'),
(10,'2024-10-31',201014.0,180912.6,2,'Aceitunas Morbe','Calle Lepanto, 8'),
(11,'2024-11-30',201019.0,180917.1,2,'Aceitunas Morbe','Calle Lepanto, 8'),
(12,'2024-12-20',128637.0,115773.3,2,'Aceitunas Morbe','Calle Lepanto, 8'),
(13,'2025-07-31',133020.0,119718.0,3,'AljaOliva S.L.','Ctra. Bormujos-Bollullos km 9'),
(14,'2025-08-31',133025.0,119722.5,3,'AljaOliva S.L.','Ctra. Bormujos-Bollullos km 9'),
(15,'2025-09-30',129010.0,116109.0,3,'AljaOliva S.L.','Ctra. Bormujos-Bollullos km 9'),
(16,'2025-10-30',138455.0,124609.5,3,'Labradores de la Campiña','Bollullos de la Mitación'),
(17,'2025-11-20',86270.0,77643.0,3,'Labradores de la Campiña','Bollullos de la Mitación');
```

5. Consultas Multitabla SQL

Aquí van las consultas que nos piden:

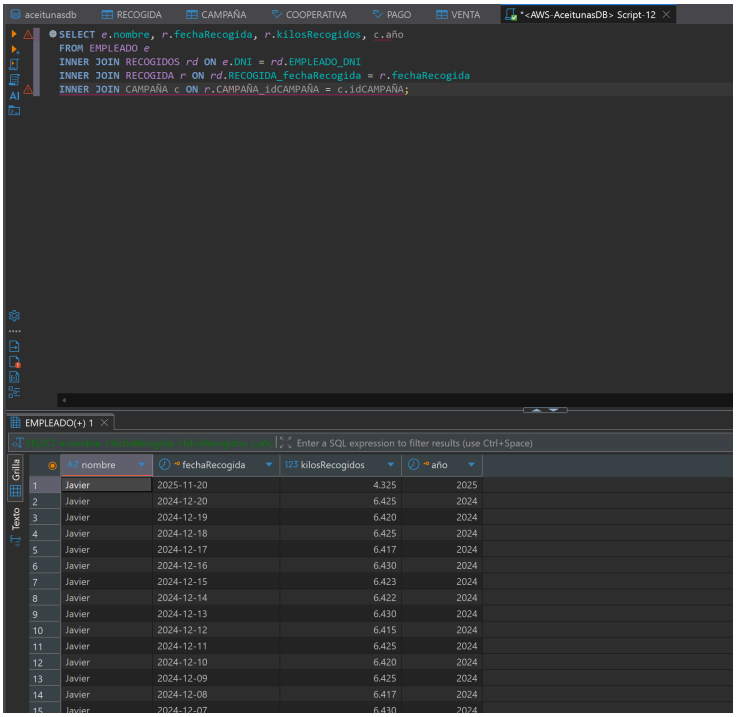
5.1 Consulta 1 – (4 tablas)

Enunciado Consulta:
Mostrar el nombre del empleado, la fecha de recogida, los kilos recogidos y el año de la campaña.

SQL:

```sql
SELECT e.nombre, r.fechaRecogida, r.kilosRecogidos, c.año
FROM EMPLEADO e
INNER JOIN RECOGIDOS rd ON e.DNI = rd.EMPLEADO_DNI
INNER JOIN RECOGIDA r ON rd.RECOGIDA_fechaRecogida =
r.fechaRecogida
INNER JOIN CAMPAÑA c ON r.CAMPAÑA_idCAMPAÑA = c.idCAMPAÑA;
```

Resultado:



"La consulta devuelve una fila por cada día de recogida.
Como todos los empleados han trabajado todos los días de la campaña, cada uno tiene 349 registros, lo que justifica la repetición de nombres en el resultado."

5.2 Consulta 2 – GROUP BY

Enunciado Consulta:
Mostrar los kilos totales recogidos por cada campaña (agrupado por año).

SQL:

```
SELECT c.año, SUM(r.kilosRecogidos) AS kilosTotales
FROM CAMPAÑA c
INNER JOIN RECOGIDA r ON c.idCAMPAÑA = r.CAMPAÑA_idCAMPAÑA
GROUP BY c.año
ORDER BY c.año;
```

Resultado:

```
SELECT c.año, SUM(r.kilosRecogidos) AS kilosTotales
FROM CAMPAÑA c
INNER JOIN RECOGIDA r ON c.idCAMPAÑA = r.CAMPAÑA_idCAMPAÑA
GROUP BY c.año;
```

AMPAÑA 1

Enter a SQL expression to filter r

	año	123 kilosTotales
1	2023	695.273
2	2024	1.111.178
3	2025	619.895

"En esta consulta se utiliza la cláusula GROUP BY para agrupar las recogidas por campaña y calcular el total de kilos recogidos en cada una de ellas mediante la función SUM."

5.3 Consulta 3 – Subconsulta

Enunciado Consulta:
Mostrar los empleados que han cobrado más que la media de pagos.

SQL:

```
SELECT DISTINCT e.nombre
FROM EMPLEADO e
INNER JOIN PAGO p ON e.DNI = p.EMPLEADO_DNI
WHERE p.importe > (
SELECT AVG(importe)
FROM PAGO
);
```

Resultado:

```
SELECT DISTINCT e.nombre
FROM EMPLEADO e
INNER JOIN PAGO p ON e.DNI = p.EMPLEADO_DNI
WHERE p.importe > (
    SELECT AVG(importe)
    FROM PAGO
);
```

MPLEADO 1

LECT DISTINCT e.nombre FROM EMPLEADO e INNER JO | Enter a

	nombre
	Javier
2	Pilar
3	Antonio
4	Lorena
5	Victoria
6	Raquel
7	Alberto
8	Rosa
9	Lidia
10	Adrian

"Como todos los empleados han cobrado el mismo importe, todos superan la media y aparecen en el resultado."

5.4 Consulta 4 – Funciones de fecha

Enunciado Consulta:
Mostrar el año y las fechas de inicio y fin de cada campaña.

SQL:

```
SELECT c.año, c.fechaInicio, c.fechaFin
FROM CAMPAÑA c;
```

Resultado:

```
SELECT c.año, c.fechaInicio, c.fechaFin
FROM CAMPAÑA c;
```

CAMPAÑA 1

	año	fechaInicio	fechaFin
1	2023	2023-07-10	2023-12-31
2	2024	2024-07-01	2024-12-20
3	2025	2025-07-01	2025-11-20

"En esta consulta se utilizan campos de tipo fecha para mostrar el inicio y el fin de cada campaña, trabajando directamente con datos de fecha sin aplicar funciones adicionales."

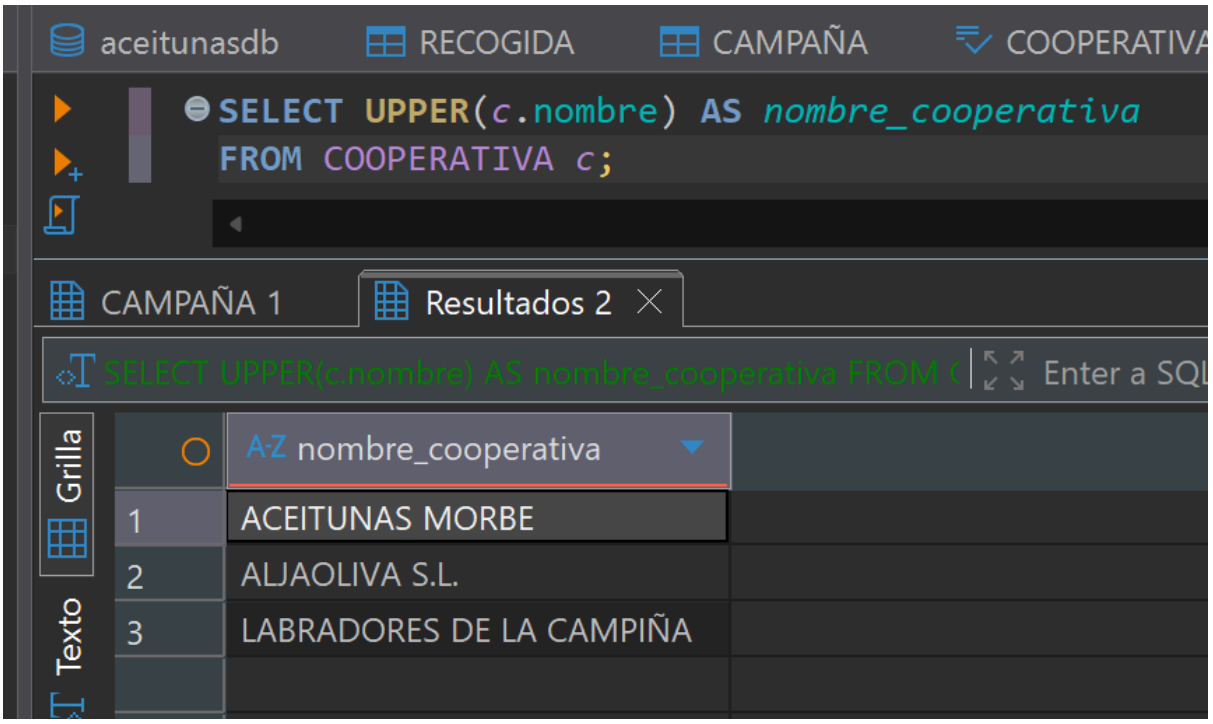
5.5 Consulta 5 – Funciones de texto

Enunciado Consulta:
Mostrar el nombre de las cooperativas en mayúsculas.

SQL:

```
SELECT UPPER(c.nombre) AS nombre_cooperativa
FROM COOPERATIVA c;
```

Resultado:



"Se utiliza la función UPPER para convertir el nombre de las cooperativas a mayúsculas."

5.6 Consulta 6 – ISNULL / IFNULL

Enunciado Consulta:
Mostrar el total pagado de cada venta y, en caso de que sea NULL, mostrar 0.

SQL:

```
SELECT idVENTA, IFNULL(totalPagado, 0) AS totalPagado
FROM VENTA;
```

Resultado:

```
SELECT idVENTA, IFNULL(totalPagado, 0) AS totalPagado
FROM VENTA;
```

MPAÑA 1 | VENTA 2

	123 idVENTA	123 totalPagado
	1	786.636
	2	1.001.934
	3	1.065.906
	4	1.107.342
	5	106.650,9
	6	1.066.284
	7	181.260
	8	1.812.546
	9	173.370,6
0	10	180.912,6
1	11	180.917,1
2	12	115.773,3
3	13	119.718
4	14	119.722,5
5	15	116.109
6	16	124.609,5
7	17	77.643

"Se utiliza la función IFNULL para mostrar el valor 0 cuando el campo totalPagado no tiene ningún valor asignado, pero todos tienen."

5.7 Consulta 7 – Consulta compleja

Enunciado Consulta:
Mostrar el total de dinero cobrado por cada empleado.

SQL:

```
SELECT e.nombre, SUM(p.importe) AS totalCobrado
FROM EMPLEADO e
INNER JOIN PAGO p ON e.DNI = p.EMPLEADO_DNI
GROUP BY e.nombre;
```

Resultado:

```
SELECT e.nombre, SUM(p.importe) AS totalCobrado
FROM EMPLEADO e
INNER JOIN PAGO p ON e.DNI = p.EMPLEADO_DNI
GROUP BY e.nombre;
```

AMPAÑA 1 | EMPLEADO 2

	nombre	totalCobrado
	Javier	29.460
2	Pilar	29.460
3	Antonio	29.460
4	Lorena	29.460
5	Victoria	29.460
6	Raquel	29.460
7	Alberto	29.460
8	Rosa	29.460
9	Lidia	29.460
10	Adrian	29.460

"Esta consulta relaciona las tablas EMPLEADO y PAGO para calcular el total de dinero cobrado por cada empleado, utilizando la función SUM y la cláusula GROUP BY, aunque todos hayan cobrado lo mismo."

6. Vistas

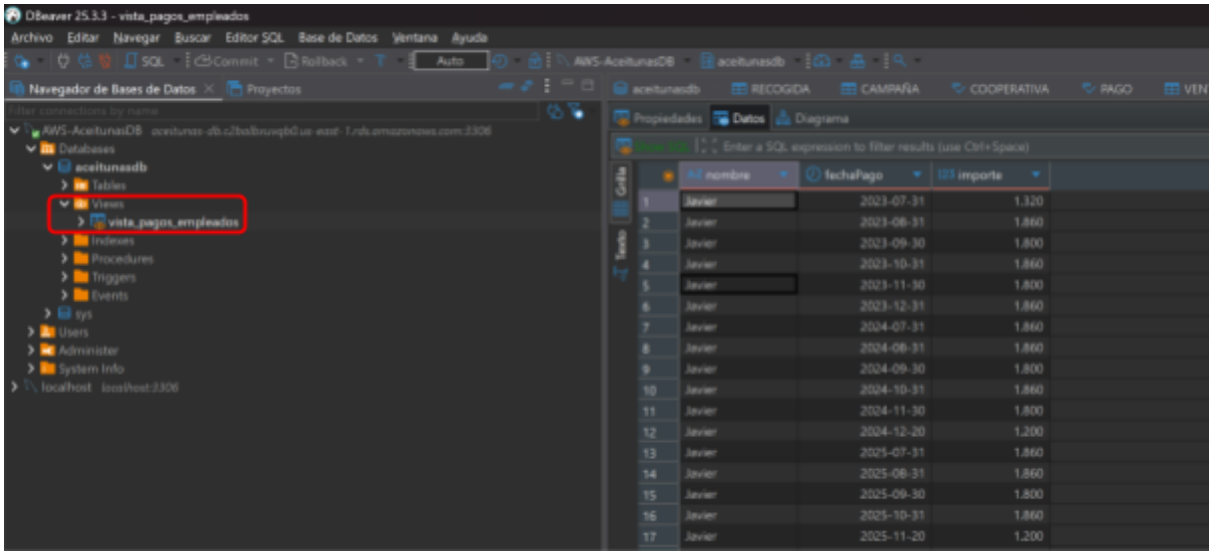
6.1 Vista 1

Enunciado de la vista:
Crear una vista que muestre el nombre del empleado, la fecha del pago y el importe.

SQL:

```sql
CREATE VIEW vista_pagos_empleados AS
SELECT e.nombre, p.fechaPago, p.importe
FROM EMPLEADO e
INNER JOIN PAGO p ON e.DNI = p.EMPLEADO_DNI;
```

Resultado:



"Se crea una vista para consultar de forma rápida los pagos realizados a cada empleado, mostrando su nombre, la fecha del pago y el importe."

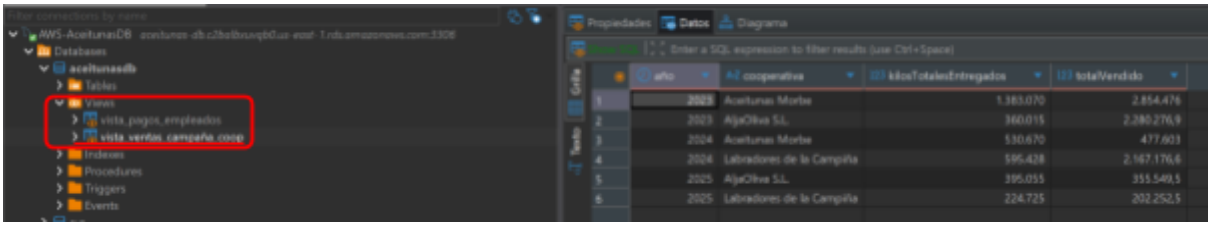
6.2 Vista 2

Enunciado de la vista:
Crear una vista que muestre el total vendido (kilos y dinero) por campaña y cooperativa.

SQL:

```
CREATE VIEW vista_ventas_campaña_coop AS
SELECT
  c.año,
  co.nombre AS cooperativa,
  SUM(v.kilosEntregados) AS kilosTotalesEntregados,
  SUM(v.totalPagado) AS totalVendido
FROM `CAMPAÑA` c
INNER JOIN VENTA v ON c.idCAMPAÑA =  .CAMPAÑA_idCAMPAÑA
INNER JOIN COOPERATIVA co ON v.COOPERATIVA_nombre = co.nombre
GROUP BY c.año, co.nombre
ORDER BY c.año, co.nombre;
```

Resultado:



"Esta vista resume las ventas agrupándolas por campaña y cooperativa, mostrando el total de kilos entregados y el total vendido, facilitando la consulta de resultados por temporada."

7. Scripts SQL del Proyecto

En este proyecto se han usado distintos scripts SQL para trabajar con la base de datos.
Estos scripts se han utilizado para hacer las consultas y para crear las vistas que se muestran en los apartados anteriores.

Todos los scripts se han ejecutado desde DBeaver sobre la base de datos alojada en AWS, por lo que no ha sido necesario incluirlos de nuevo en este apartado.
Por ejemplo de carga masiva → 4. Carga Masiva de Datos
o también de consultas → 5. Consultas Multitabla SQL
O de vistas → 6. Vistas

8. Conclusión y valoración personal Parte 1

Este proyecto me ha ayudado a entender mejor cómo funcionan las bases de datos en la práctica. No solo he diseñado la base de datos y las tablas, sino que también he trabajado con datos reales, cargándola de datos mediante archivos CSV y scripts SQL, lo que me ha servido para ver cómo se manejan grandes cantidades de información.

Además, he realizado distintas consultas SQL utilizando INNER JOIN, GROUP BY, subconsultas y funciones básicas, lo que me ha permitido sacar información útil de la base de datos y comprobar que los datos estaban bien relacionados. También he creado vistas para facilitar la consulta de los pagos y de las ventas por campaña.

En general, ha sido un proyecto útil porque me ha permitido aplicar lo aprendido en clase a un caso real, como la gestión de la recogida de aceitunas, y me ha ayudado a reforzar conceptos que me servirán en segundo de DAW y en trabajos más grandes.